Structures

.... and other user-defined data types



INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Basic Definitions

What is a Structure?

It is a convenient construct for representing a group of logically related data items.

- Examples:
  - Student name, roll number, and marks.
  - Real part and complex part of a complex number.

Structures help in organizing complex data in a more meaningful way.

The individual structure elements are called *members*.

This is our first look at a user-defined data-type.

Defining a Structure

The composition of a structure may be defined as:

```
struct ⟨ name of structure ⟩ {
        ⟨ data-type ⟩ ⟨ member-name1⟩ ;
        ⟨ data-type ⟩ ⟨ member-name2⟩ ;
        …
        ⟨ data-type ⟩ ⟨ member-namek⟩ ;
};
```

For example:

```
struct point  {
        float xcoord;
        float ycoord;
};
```

Example

A structure definition

```
struct student {
        char name[30];
        int  roll_number;
        int  total_marks;
        char dob[10];
      };
```

Defining structure variables:

```
struct student  a1, a2, a3;
```

A new data-type

- struct is the required C keyword
- Do not forget the ; at the end of the structure defn
- The individual members can be ordinary variables, pointers, arrays, or other structures (any data-type)
- The member names within a particular structure must be distinct from one another
- A member name can be the same as the name of a variable defined outside of the structure

Structure Definition versus Structure Variable Declaration

Structure Definition

```
struct point  {
        float xcoord;
        float ycoord;
};
```

- No memory is allocated
- Only defining a new data-type

Structure Variable Declaration

```
struct point a, b, c;
```

- Here a, b, c are variables of the type struct point
- Memory is allocated for a, b, c.
- Variable declaration is possible *after* definition

Each structure type variable (e.g., a, b, c) has its own copy of each member of that structure type

Structure Variable Declaration can be clubbed with Definition

Separately:

```
struct point  {
        float xcoord;
        float ycoord;
};

struct point  a, b, c;
```

Together:

```
struct point  {
        float xcoord;
        float ycoord;
} a, b, c;
```

- The struct definition can be reused elsewhere
- Like:

  struct point p, q;

Another way:

```
struct  {
        float xcoord;
        float ycoord;
} a, b, c;
```

- In this case we do not have a name for the struct
- Hence we cannot reuse the struct definition

Each structure type variable (e.g., a, b, c) has its own copy of each member of that structure type

Type Definitions

The *typedef* construct

The *typedef* construct can be used to define new (derived) data types in C.

```
typedef  float  kilometers_per_hour;        // kilometers_per_hour is a new data type
                                            // Note that no variable is allocated space here

kilometers_per_hour speed;                  // Here speed is a declared variable
speed = 40;
```

Structures and *typedef*

Without *typedef*

```
struct  complex
{
    float  real;
    float  imag;
} ;

struct complex a, b, c;
```

Here struct complex is like a new data type.

With *typedef*

```
typedef  struct
{
    float  real;
    float  imag;
} complex ;

complex a, b, c;
```

Here complex is a new data type

Accessing members and using structures

Accessing the members of a structure

- The members of a structure are accessed individually, as separate entities.

- A structure member can be accessed by writing

  ⟨variable-name⟩.⟨member-name⟩

  where *variable* refers to the name of a structure-type variable, and *member* refers to the name of a member within the structure.

Dot operator used to access members of a structure, through a structure-type variable

```
struct point  {
        float xcoord;
        float ycoord;
} a, b;
a.xcoord = 2.5;    a.ycoord = 3.2;
b.xcoord = b.ycoord = 0;
```

Structure initialization

Structure variables may be initialized following similar rules of an array. The values are provided within the second braces separated by commas

An example:

```
struct complex a={1.0,2.0}, b={-3.0,4.0};
```



```
a.real=1.0;   a.img=2.0;
b.real=-3.0;  b.img=4.0;
```

Example: Addition of two complex numbers

```
#include  <stdio.h>
main( )
{
        struct  complex
        {
                float  real;
                float  imag;
        }  a, b, c;

        scanf (“%f %f”, &a.real, &a.imag);
        scanf (“%f %f”, &b.real, &b.imag);

        c.real = a.real + b.real;
        c.imag = a.imag + b.imag;
        printf (“\n %f  + %f j”, c.real, c.imag);
}
```

Structure declaration can be outside main() as well. This is necessary if a program has multiple functions using a structure type.

Assignment of Structure Variables

```
struct class
{
      int  number;
      char name[20];
      float marks;
};
```

```
main()
  {
      int  x;
      struct class student1 = {111, "Rao", 72.50};
      struct class student2 = {222, "Reddy", 67.00};
      struct class student3;

      student3 = student2;
  }
```

A structure variable can be directly assigned to another variable of the same type.
All the individual members get assigned / copied.

But, two structure variables can NOT be compared for equality or inequality

if (student1 == student2)…… this cannot be done

An interesting observation

```
int a[5] = { 10, 20, 30, 40, 50 };
int b[5];

b = a;
```

X This is not allowed

```
struct list {
    int x[5];
};

struct list a, b;
a.x[0] = 10;   a.x[1] = 20;
a.x[2] = 30;   a.x[3] = 40;
a.x[4] = 50;

b = a;
```

This is allowed !!

Structures can be copied directly – even if they contain arrays !!

Arrays of structures

Arrays of Structures

Once a structure data-type has been defined, we can declare an array of structures.

```
struct class
{
    int  number;
    char name[20];
    float marks;
};
struct class student[50];
```

- The individual members can be accessed as:

student[k].marks	*marks of the kth student*
student[k].name	*name of the kth student*
student[k].name[j]	*j^{th} character in the name of the kth student*

Example: Store a list of students (name, CGPA), compute average CGPA

```
#include <stdio.h>

struct student{
      float cgpa;
      char name[10];
};
int main( ){
      int i; float avg;
      struct student st[5];
      printf("Enter records of 5 students");
      for(i=0;i<5;i++){
        printf("\nEnter Cgpa:");
        scanf("%f",&st[i].cgpa);
        printf("\nEnter Name:");
        scanf("%s",st[i].name);
      }
```

```
    // compute average cgpa
    avg = 0.0;
    for(i=0;i<5;i++)
        avg += st[i].cgpa;

    avg = avg / 5.0;
    printf("Avg cgpa:%f",avg);

    return 0;
}
```

Structures and functions

Structures are passed by value to functions

```
#include <stdio.h>

typedef struct {
        float real;
        float imag;
} _COMPLEX;

void swap ( _COMPLEX a,  _COMPLEX b)
{
   _COMPLEX tmp;

   tmp = a;   a = b;   b = tmp;
 }
```

```
void print ( _COMPLEX a)
{
    printf("(%f, %f) ", a.real, a.imag);
}

main( )
{
    _COMPLEX x = { 4.0, 5.0 }, y = { 10.0, 15.0 };

    print(x); print(y); printf("\n");
    swap(x, y);
    print(x); print(y); printf("\n");
}
```

Program output:

(4.000000, 5.000000) (10.000000, 15.000000)
(4.000000, 5.000000) (10.000000, 15.000000)

No swapping takes place actually, similar to what we saw for integers, floats, etc.

Structures can be returned from functions

```
#include <stdio.h>

typedef struct {
        float real;
        float imag;
} _COMPLEX;


_COMPLEX add ( _COMPLEX a,  _COMPLEX b)
{
    _COMPLEX tmp;
    tmp.real = a.real + b.real;
    tmp.imag = a.imag + b.imag;
    return tmp;
 }
```

```
main( )
{
    _COMPLEX x = { 4.0, 5.0 }, y = { 10.0, 15.0 };
    _COMPLEX z;

    z = add(x, y);
    printf(" %f, %f \n", z.real, z.imag);
}
```

Program output:
14.000000, 20.000000

Pointers to Structures

Pointers and Structures

```
struct class
 {
      int  roll;
      char name[20];
      float marks;
 };
```

struct class* ptr;

Once `ptr` points to a structure variable, the members can be accessed as:

```
ptr -> roll;
ptr -> name;
ptr -> marks;
```

- The symbol "–>" is called the *arrow* operator.

Arrow operator used to access members of a structure, through a structure-type pointer.

Use of pointers to structures

```
#include <stdio.h>
struct complex  {
                float  real;
                float  imag;
};

main( )
{
   struct  complex  a, b, c;
   scanf ( "%f %f", &a.real, &a.imag );
   scanf ( "%f %f", &b.real, &b.imag );
   add( &a, &b, &c ) ;
   printf ( "\n %f %f", c,real, c.imag );
}
```

```
void add (struct complex* x, struct complex* y,
struct complex* t)
{
   t->re = x->real + y->real;
   t->im = x->imag + y->imag;
}
```

A Warning

When using structure pointers, we should take care of operator precedence.

- Member operator "." has higher precedence than "*"

  ptr –> roll and (*ptr).roll mean the same thing.

  *ptr.roll will lead to error.

- The operator "–>" enjoys the highest priority among operators.

  ++ptr –> roll will increment roll, not ptr.

  (++ptr) –> roll will increment the pointer (to the next structure in an array) and access roll in the next structure.

Practice problems

1. Extend the complex number program to include functions for addition, subtraction, multiplication, and division
2. Define a structure for representing a point in two-dimensional Cartesian co-ordinate system. Using this structure for a point
   i. Write a function to return the distance between two given points
   ii. Write a function to return the middle point of the line segment joining two given points
   iii. Write a function to compute the area of a triangle formed by three given points
   iv. Write a main function and call the functions from there after reading in appropriate inputs (the points) from the keyboard

3. Define a structure STUDENT to store the following data for a student: name (null-terminated string of length at most 20 chars), roll no. (integer), CGPA (float). Then
    1. In main, declare an array of 100 STUDENT structures. Read an integer n and then read in the details of n students in this array
    2. Write a function to search the array for a student by name. Returns the structure for the student if found. If not found, return a special structure with the name field set to empty string (just a '\0')
    3. Write a function to search the array for a student by roll no.
    4. Write a function to print the details of all students with CGPA > x for a given x
    5. Call the functions from the main after reading in name/roll no/CGPA to search